

Padding Oracle Attack for Dummies

A beginner's journey from encryption to decryption without the key

"All you need to master something is TIME + willpower (time to learn and willpower to keep answering your questions and deepening your understanding)"

— Oussama Mejdoubi

A complete, structured guide

Contents

1	Real-World Context: How Modern Encryption Works	2
2	What is AES? (The Encryption Algorithm)	3
3	What is CBC Mode? (Chaining Blocks Together)	4
4	Padding: Making Messages Fit	5
5	Encryption in Action: From User to Server	7
6	Decryption: The Server's Side	8
7	The Information Leak: What Goes Wrong	9
8	The Padding Oracle: Asking Yes/No Questions	10
9	Technical Deep Dive: The Algorithm Step-by-Step	11
10	Putting It Together: The Complete Attack	13

Real-World Context: How Modern Encryption Works

Before we talk about the attack, let's understand what we're attacking. Almost every encrypted message you send — emails, passwords, bank transactions — goes through this pipeline.

The journey of an encrypted message

Your browser or app wants to send a secret message to a server. Here's what happens:

```
graph TD
    A[User has message] --> B[Message gets encrypted (using AES)]
    B --> C[Encrypted data sent over network]
    C --> D[Server receives encrypted data]
    D --> E[Server decrypts it (using AES)]
    E --> F[Server reads the message]
    F --> G[Server sends response (encrypted)]
```

The key question: If someone intercepts the encrypted message, can they read it? No. Without the encryption key, it's gibberish. Unless... there's a flaw in how the decryption works.

What we'll discover: A tiny flaw — the server accidentally leaking information about whether a decrypted message's padding is valid — is enough to decrypt any message, even without the key.

Why this matters

This isn't theoretical. Real systems have been vulnerable:

- **ASP.NET (2010):** Microsoft emergency patch after attackers exploited this flaw against web applications.
- **TLS/SSL (2011-2013):** SSL libraries leaked timing information that exposed the same vulnerability.
- **Today:** Any system using CBC mode encryption (still common) without proper safeguards is at risk.

What is AES? (The Encryption Algorithm)

AES is a symmetric encryption algorithm — it means the same key encrypts and decrypts. Think of it like a mechanical lock.

AES as a black box

Imagine you have a magic box:

```
INPUT: "hello world" + KEY
↓
[AES BOX]
↓
OUTPUT: "g3x2k9p1m5d8..." (gibberish)
```

The magic box (AES) takes plaintext and a key, outputs ciphertext. Without the key, you cannot decrypt.

Key insight: AES encrypts one block at a time . To encrypt larger messages, you need a mode . That's where CBC comes in.

What is CBC Mode? (Chaining Blocks Together)

CBC = Cipher Block Chaining. It's a method to chain multiple blocks together so patterns don't repeat.

Why do we need modes?

If you encrypted two identical messages with AES alone, you'd get identical ciphertexts. That's bad — an attacker can see patterns. CBC solves this.

How CBC encryption works (simplified)

Imagine encrypting a 16-byte message "helloworld12345":

```
STEP 1: Create random 16-byte IV (Initialization Vector)
IV = "randomrandom1234"
```

```
STEP 2: Mix plaintext with IV
plaintext  $\oplus$  IV = "mixed_stuff"
```

```
STEP 3: Encrypt the mixed stuff with key
AES_encrypt("mixed_stuff", key) = "ciphertext_block"
```

```
OUTPUT: IV + ciphertext_block (both can be sent openly)
```

Key point: The IV is random, so same plaintext + same key = different ciphertext each time. Secure!

For decryption (the reverse): The server receives IV + ciphertext, then: 1. Decrypt ciphertext with key → get the mixed stuff 2. Mix with IV again → get original plaintext

Padding: Making Messages Fit

AES works with 16-byte blocks. What if your message is only 10 bytes? You need padding — extra bytes to fill the block. The clever part: each padding byte tells you how many padding bytes there are.

PKCS#7 padding rule

Append N bytes, where each byte has value N (the number of padding bytes needed). This way, the server can read the last byte and immediately know how many padding bytes to remove.

Case 1: Message needs 1 byte of padding (0x01)

```
Message: "abcdefghijklmno" (15 bytes)
Need: 16 - 15 = 1 byte padding
Padding: 1 byte with value 0x01

FINAL: "abcdefghijklmno" + [0x01] = 16 bytes
```

On decryption: server reads last byte = 0x01, removes 1 byte, gets original message.

Case 2: Message needs 11 bytes of padding (0x0B)

```
Message: "hello" (5 bytes)
Need: 16 - 5 = 11 bytes padding
Padding: 11 bytes, each with value 0x0B

FINAL: "hello" + [0x0B × 11] = 16 bytes
```

On decryption: server reads last byte = 0x0B (11 in decimal), removes 11 bytes, gets original message.

Case 3: Message needs 7 bytes of padding (0x07)

```
Message: "password" (8 bytes)
Need: 16 - 8 = 8... wait, let me recalculate
Need: 16 - 9 = 7 bytes padding
Padding: 7 bytes, each with value 0x07

FINAL: "password" + [0x07 × 7] = 16 bytes
```

Pattern: If you need N bytes of padding, append N bytes where each byte = 0x0N (N in hex). Value 1 = 0x01, value 11 = 0x0B, value 7 = 0x07, etc.

! The tricky case: Message is exactly 16 bytes (block is FULL)

This seems simple — the message is already complete! But here's the problem:
The ambiguity problem:

Without a full padding block: Message: "complete16bytes!" (16 bytes) No padding added → 16 bytes total

Problem on decryption: Server reads last byte = 0x21 (ASCII '!') Does this mean: "remove 33 bytes of padding" or "this is the message"? ✗ CONFUSION!

Solution: Always add a whole new padding block! Message: "complete16bytes!" (16 bytes) Padding: $[0x10 \times 16] = 16$ bytes of padding (value 16 = 0x10) FINAL: 32 bytes total

Why this works: On decryption, the server reads the last byte. If it's 0x10 (16), the server knows: "remove 16 bytes (the entire last block)". Now there's no confusion!

Summary: All major padding cases

Message length: 1 byte — Padding: $[0x0F \times 15] \rightarrow 16$ bytes total

Message length: 8 bytes — Padding: $[0x08 \times 8] \rightarrow 16$ bytes total

Message length: 15 bytes — Padding: $[0x01 \times 1] \rightarrow 16$ bytes total

Message length: 16 bytes — Padding: $[0x10 \times 16] \rightarrow 32$ bytes total !

! Critical: If the server receives a message with invalid padding, it throws an error. This tiny error message is the start of our attack.

Encryption in Action: From User to Server

Let's walk through a real example: a user sending a password to a server.

Example: Encrypting a password

User's browser wants to send password "MyPass2024" to server. Here's what happens:

- **Create message with padding:** "MyPass2024" is 10 bytes. Needs 6 bytes padding (value 0x06 each). Original: "MyPass2024" Padded: "MyPass2024" + [06 06 06 06 06 06] = 16 bytes
- **Generate random IV:** Browser creates 16 random bytes. IV = "aB#\$xY%&@!~zQ9mK" (16 random bytes)
- **Mix with IV:** XOR (mix) padded message with IV. $\text{padded_msg} \oplus \text{IV} = \text{"mixed_result"}$
- **Encrypt:** AES encrypts the mixed result using the secret key. $\text{AES_encrypt("mixed_result", SECRET_KEY)} = \text{"k3x9p2m1d5w8..."}$
- **Send:** Browser sends [IV + encrypted_data] to server in plaintext (no further encryption). Send: "aB#\$xY%&@!~zQ9mK" + "k3x9p2m1d5w8..."

Key insight: The IV doesn't need to be secret — it's random. Only the SECRET_KEY is confidential.

Decryption: The Server's Side

The server receives [IV + encrypted_data] and decrypts. This is where the vulnerability hides. Let's see exactly what happens, byte by byte.

Complete decryption flow (byte-by-byte visualization)

What the server receives and how it processes it: **Key observation:** The server returns ✓ VALID or ✗ INVALID . This response is what the attacker exploits.

What if padding is invalid?

Same process, but at Step 3 the check fails:

! The vulnerability: The server's response — "valid" or "invalid" padding — leaks information about the decrypted bytes. An attacker uses this to reverse-engineer the plaintext.

The Information Leak: What Goes Wrong

The server is supposed to be secure. But by accident, it tells the attacker: "that padding is invalid" or "that padding is valid". This tiny piece of information is dangerous.

Why does the server leak this?

The server has to check padding. There are only two outcomes:

✓ Padding is valid > or > ✗ Padding is invalid

The problem: An attacker can send modified ciphertext and observe which outcome happens. This gives information.

How the leak happens in practice

Different systems leak information in different ways:

The "oracle": The attacker calls this the padding oracle — an oracle because it answers yes/no questions about padding validity.

The Padding Oracle: Asking Yes/No Questions

An attacker can send modified ciphertext to the server and learn: "Is this padding valid?" By asking this question 256 times, they can recover encrypted bytes.

Simple example: Finding one encrypted byte

The attacker has an encrypted message. They want to know: what is the last encrypted byte? **The attacker's process:**

Step 1: Send the ciphertext with last byte modified to 0x00. Server: "✗ Invalid padding"

Step 2: Send the ciphertext with last byte modified to 0x01. Server: "✗ Invalid padding"

Step 3: Try 0x02, 0x03, 0x04... 0xFF Server keeps saying: "✗ Invalid padding"

Step 4: Try 0xD8. Server: ✓ Valid padding!

Success! The attacker found that when last byte = 0xD8, padding is valid. This means something specific about the decrypted message.

Why this works: The attacker modified the ciphertext but NOT the message inside. They triggered a specific condition in the decrypted data: valid padding. This condition happens for exactly one byte value (out of 256). Finding it took 256 tries.

What the attacker learned

The attacker now knows: when the last ciphertext byte = 0xD8, the decrypted message ends with valid padding of type "01" (one byte of value 0x01). This tells them something about the encrypted byte. Using algebra (which we'll skip for now), they can calculate the original encrypted value.

The magic: By repeating this 16 times (once per byte in a block), the attacker recovers the entire decrypted block — without the encryption key!

Technical Deep Dive: The Algorithm Step-by-Step

Now let's get rigorous. The padding oracle attack is built on one principle: **we build assumptions and verify them**. Here's exactly how, with the tricky ambiguity problem included.

Core Principle: Assume & Verify

- 1. Build assumption:** "If I set $IV[\text{last}] = 0xD8$, the plaintext will end with $0x01$ "
- 2. Test it:** Send modified ciphertext to the oracle
- 3. Verify:** Oracle returns VALID or INVALID
- 4. Solve:** Use response + your chosen IV to extract the unknown encrypted byte

The Attack Equation (Everything Hinges On This)

CBC decryption, for every byte position i :

$$P_1[i] = D(C_1)[i] \oplus IV[i]$$

What you CAN control: the $IV[i]$ — you can set this to any of 256 values, and you want to engineer $P_1[i] = 0x01$ (or $0x02$, $0x03$, etc.).

What you CANNOT know directly: $D(C_1)[i]$, the encrypted intermediate state (unknown but FIXED). The oracle only tells you VALID or INVALID.

The magic: by controlling IV and observing the oracle response, you can deduce $D(C_1)[i]$ using pure algebra.

Step-by-Step: Recovering Byte 7 (The Last Byte)

Block size = 8 bytes. We target the last byte first. Original plaintext: `hello\x03\x03\x03`.

- 1. Brute-force the IV:** Modify ONLY $IV[7]$; keep $IV[0-6]$ unchanged. Try all 256 values: $IV'[7] = 0x00, 0x01, \dots, 0xFF$. Send each modified ciphertext to the server.
- 2. Oracle returns VALID:** After ~ 256 tries, suppose $IV'[7] = 0xD8$ returns $\checkmark$ VALID. This means: when $IV[7] = 0xD8$, the plaintext ends with valid padding.
- 3. Ambiguity Problem:** What does "valid" actually mean? Case A: you created $0x01$ padding. Case B: you accidentally preserved the original $0x03\ 0x03\ 0x03$. BOTH return VALID — you can't tell them apart yet.
- 4. Verification Test (the key trick):** Change another byte too — set $IV'[6] = 0x99$ — and resend. If **still VALID** $\rightarrow$ padding was $0x01$ (only the last byte matters). If **now INVALID** $\rightarrow$ padding was $0x03\ 0x03\ 0x03$ (a false positive).
- 5. Solve the Algebra:** You've confirmed $D(C_1)[7] \oplus 0xD8 = 0x01$, so $D(C_1)[7] = 0x01 \oplus 0xD8 = \mathbf{0xD9}$.

6. Recover Plaintext (using original IV): $P_1[7] = D(C_1)[7] \oplus \text{original_IV}[7] = 0xD9 \oplus 0x00 = \mathbf{0xD9}$. One byte recovered!

Why XOR Reversibility is the KEY

The entire attack depends on this mathematical property:

XOR is self-reversing: If $A \oplus B = C$, then $A = B \oplus C$ (and also $B = A \oplus C$).

Example: $0xD9 \oplus 0xD8 = 0x01$, therefore $0xD9 = 0x01 \oplus 0xD8$. This is how we extract the intermediate byte without ever knowing the key.

Repeat for All Remaining Bytes

Once byte 7 is done, target byte 6, then 5, and so on.

```
Byte 6 (target 0x02 0x02 padding):
Set IV'[7] = D(C1)[7] ⊕ 0x02 (to force last byte = 0x02)
Brute-force IV'[6] until VALID
Solve: D(C1)[6] = (found_value) ⊕ 0x02

Byte 5 (target 0x03 0x03 0x03 padding):
Set IV'[7] and IV'[6] to match
Brute-force IV'[5] until VALID
Solve: D(C1)[5] = (found_value) ⊕ 0x03
```

Continue this pattern. Max 256 tries per byte = $\sim 2,048$ requests total per block.

The False Positive Problem (The Subtle Part)

When the original plaintext already has valid padding, you get ambiguity.

Example: Original plaintext = "hello\x03\x03\x03"

When you brute-force IV[7], you might find TWO valid values:

IV'[7] = 0xD8 → creates plaintext ending 0x01 → ✓ VALID

IV'[7] = 0x03 → preserves plaintext ending 0x03 0x03 0x03 → ✓ VALID

Both return VALID! This is the ambiguity.

Solution: the verification test (changing byte 6) tells you which is real.

Note: to verify this assumption we built a Python script as proof. You can find it here: <https://github.com/sam99235/padding-oracle-attack>

The elegance of the attack: Build assumptions (modify IV to engineer plaintext). Verify them (oracle response). Extract knowledge (XOR reversibility). Distinguish false positives (second-byte test). Repeat for all bytes. The server's padding check, meant to protect you, leaks the information that decrypts the whole message.

Putting It Together: The Complete Attack

NOTE: In this demo we are using **ONE** ciphertext. The same principle applies for other padded ciphertexts — just choose a ciphertext where the padding exists.

Now you understand each piece. Let’s see exactly how an attacker decrypts a message — byte by byte — using nothing but the oracle’s yes/no answers. We’ll use a real decryption table to make every step visible.

The Setup: What the Attacker Sends

The attacker captures one ciphertext block and sends it to the server with an IV of all zeros. Here is the full picture of what happens inside the server’s decryption routine:

Attacker sends 1 ciphertext block + IV of all 0x00								
Ciphertext = \x7c\xd3\xdc\x3e\x98\xcb\x52 ... \x81								
Ciphertext C ₁	7c	d3	dc	3e	98	cb	52	81
↓ Decryption Routine with Secret Key ↓								
D(C ₁) (UNKNOWN)	4f	9d	e0	f6	9a	7a	20	d9
XOR	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
IV (you control)	00	00	00	00	00	00	00	00
= Plaintext P ₁ = D(C ₁) ⊕ IV =								
P ₁ (UNKNOWN)	4f	9d	e0	f6	9a	7a	20	d9

■ byte under attack ■ recovered (only 8 of 16 columns shown to fit the page)

The result: when the last plaintext byte isn’t valid padding, the server responds **“Invalid Padding.”** That single response is the entire leak the attacker needs.

Before the Attack: What Do We Actually Know?

- WE CONTROL** — the IV. We can set any byte to any value (0x00-0xFF) and resend.
- WE DON’T KNOW** — the intermediate state D(C₁). Fixed (the ciphertext doesn’t change), but hidden because we don’t have the key.
- WE DON’T KNOW** — the plaintext P₁. That’s the secret we’re trying to steal.
- WE LEARN** — one bit per request: did the padding check pass or fail?

The whole trick: D(C₁) is fixed and IV is ours to choose. So if we can force the plaintext to become valid padding by tweaking the IV, the relationship between our IV and the known padding value reveals the hidden D(C₁) byte.

The Secret Weapon: XOR is Reversible

Everything depends on one property of XOR. It works exactly like addition and subtraction cancel out:

Addition: $5 + 2 = 7 \rightarrow 7 - 2 = 5$

XOR: $x \oplus a = b \rightarrow x = b \oplus a$

Three properties make it perfectly reversible: **commutative** ($x \oplus y = y \oplus x$), **self-inverse** ($x \oplus x = 0$, $x \oplus 0 = x$), **associative** ($(x \oplus y) \oplus z = x \oplus (y \oplus z)$). So if we know $P_1[i]$ (forced valid padding) and $IV[i]$ (we chose it):

$$D(C_1)[i] = P_1[i] \oplus IV[i]$$

How Padding Hands Us the Answer

The attacker doesn't know the plaintext — but they know the *rules* of valid padding. When the oracle says “valid” for the last byte, there's only one thing it can mean:

Oracle says VALID $\rightarrow P_1[15] = 0x01$

Not “some padding.” Just **exactly 0x01** — the only valid 1-byte padding. The attacker has manufactured a known plaintext byte:

Worked example — recovering the last byte

1. Brute-force $IV[15]$: try 0x00, 0x01, ... until the oracle says VALID. Suppose $IV[15] = 0xD8$ works.

2. We now know: $D(C_1)[15] \oplus 0xD8 = 0x01$

3. Reverse with XOR: $D(C_1)[15] = 0x01 \oplus 0xD8 = \mathbf{0xD9}$

4. Recover plaintext using the *original* IV (say it was 0x00): $P_1[15] = 0xD9 \oplus 0x00 = \mathbf{0xD9}$

That 0xD9 didn't come from a guess. It came from pure algebra. The attacker recovers the intermediate state first, then applies the real IV to get the actual plaintext.

Testing & Verification: Avoiding False Alarms

What if the original message already ended in valid padding — like `hello\x03\x03\x03`? Then brute-forcing the last IV byte can produce **two** “valid” responses, and one is a trap. **The ambiguity**

Real hit: your IV created plaintext ending in **0x01** $\rightarrow$ VALID

False positive: your IV accidentally recreated the original **0x03 0x03 0x03** $\rightarrow$ also VALID

The verification test: change a *different* byte — say the second-to-last IV byte — and resend.

- **Stays VALID** → padding was **0x01** (only the last byte is checked). Real hit confirmed.
- **Turns INVALID** → padding was **0x03 0x03 0x03** (needs three matching bytes, you broke one). False positive — discard it.

Assume, then verify. This is the heartbeat of the attack: assume a valid response means 0x01 padding, then verify by perturbing another byte. If the assumption survives, the byte is real and the algebra holds.

Recovering the Whole Block

Once the last byte is solved, you march leftward. For each new byte you target a longer padding value, forcing the already-known bytes to match:

Progression of targeted padding				
Target byte 15	...	...	...	01
Target byte 14	...	...	02	02
Target byte 13	...	03	03	03
Target byte 12	04	04	04	04

■ currently brute-forcing ■ already recovered (forced to match)

Each byte needs at most 256 tries. For a 16-byte block that's ~4,096 requests to fully decrypt — without ever touching the key.

Final recovered plaintext: "MyPass2024" + [06 06 06 06 06 06]
Strip padding → "MyPass2024"
Password stolen.

Remarkable fact: the attacker never needs the encryption key and never breaks AES. They only need the ciphertext, the ability to resend modified versions, and the server's one-bit yes/no answer about padding.

Why this is called a “side-channel” attack

AES itself stays unbroken. The attacker exploits a side-effect — the server leaking padding validity through error messages, timing, or response size. It's like cracking a safe not by forcing the lock, but by listening to the clicks as the dial turns.

Real-world impact: In 2010, Microsoft shipped an emergency patch (MS10-070) because ASP.NET was vulnerable to exactly this attack, putting thousands of web apps at risk.

How to defend against it

Use AEAD encryption: modes like AES-GCM or ChaCha20-Poly1305 authenticate the ciphertext. Modify one bit and decryption fails outright — no padding step to leak from.

Or: make valid and invalid padding indistinguishable — identical responses and constant-time comparison, so there's no signal to measure.